TECHNICAL INTERVIEW · KNOWLEDGE BASE

Full-Stack Engineer Technical Q&A Prep

Answer bank for the technical questions likely to come up in the SnT / Wingos interview. Focused on **demonstrating knowledge**, not recalling your CV.

Covers: **JavaScript / TypeScript** · **React & Next.js** · **Node.js** · **Python / FastAPI** · **PostgreSQL / MySQL / MongoDB** · **Git · Docker · CI/CD** · **LLM integration** · **Performance · Reliability · Security** · **System & API design**

1 JavaScript & TypeScript

Language fundamentals interviewers use to gauge depth quickly.

Q Explain the event loop. How do the call stack, microtask queue, and macrotask queue interact?

JS runs single-threaded. Synchronous code runs on the **call stack**. When it's empty, the loop drains the **microtask queue** fully (Promise callbacks, `queueMicrotask`, `await` continuations), then takes **one** macrotask (`setTimeout`, I/O, UI events), then drains microtasks again.

Key consequence: microtasks always run before the next timer, so a Promise chain resolves before a `setTimeout(…, 0)` queued earlier.

> **Likely follow-up:** "What logs first?" — a `Promise.resolve().then()` vs a `setTimeout(0)`. Answer: the Promise, because microtasks drain before the next macrotask.

Q What's the difference between `==` and `===`, and when is `==` ever acceptable?

`===` compares type and value with no coercion. `==` coerces. The one idiomatic use of `==` is `x == null`, which is true for both `null` and `undefined` — a concise nil check. Otherwise default to `===`.

Q Closures — what are they and give a real use.

A closure is a function retaining access to variables from the scope it was defined in, even after that scope returns. Practical uses: data privacy (module pattern), memoization caches, and stable callback state — e.g. a `once()` wrapper or a debounce holding its timer in the enclosing scope.

Q `var` vs `let` vs `const`; explain hoisting and the temporal dead zone.

`var` is function-scoped and hoisted as `undefined`. `let` / `const` are block-scoped and hoisted but uninitialized — accessing them before declaration throws (the **temporal dead zone**). `const` forbids reassignment but the referenced object is still mutable.

Q How does prototypal inheritance work?

Each object has an internal link (`[[Prototype]]`) to another object. Property lookup walks this chain until found or it hits `null` . `class` is syntactic sugar over this — methods live on `Prototype.prototype` , shared across instances rather than copied.

Q What does `this` refer to, and how do arrow functions change it?

`this` is determined by **how a function is called**: method call → the object; plain call → `undefined` (strict) or global; `new` → the new instance; `call/apply/bind` → explicit. Arrow functions have **no own** `this` — they capture it lexically from the enclosing scope, which is why they're ideal for callbacks inside methods.

Q Deep vs shallow copy — how do you handle each?

Shallow (`{...obj}` , `Object.assign`) copies top level; nested references are shared. Deep copy: `structuredClone(obj)` is the modern built-in (handles Dates, Maps, cyclic refs). `JSON.parse(JSON.stringify())` works but drops functions, `undefined` , and mangles Dates.

Q TypeScript: `interface` vs `type` . When do you pick which?

- **interface** — object/class shapes; supports declaration merging and `extends` . Idiomatic for public API contracts.
- **type** — everything interfaces can't: unions, intersections, tuples, mapped/ conditional types, primitives.

Rule of thumb: interface for object shapes, type when you need unions or computed types.

Q Explain generics with a quick example.

Generics parameterize types so a function keeps type info instead of collapsing to `any` .

```
function first<T>(arr: T[]): T | undefined {
  return arr[0];
}
const n = first([1, 2, 3]); // n: number
```

Q What are `unknown`, `never`, and `any` — and why avoid `any`?

- `any` disables checking — it's an escape hatch that spreads unsafety.
- `unknown` is the type-safe top type: you must narrow before use.
- `never` is the bottom type — no value; used for exhaustiveness checks and functions that never return.

Q Useful utility types you actually reach for?

`Partial<T>`, `Required<T>`, `Pick<T,K>`, `Omit<T,K>`, `Record<K,V>`, `Readonly<T>`, and `ReturnType<F>`. Being able to name a couple with a use case (e.g. `Omit` to build a DTO from an entity) signals real usage.

2 React & Next.js

The frontend framework the role centres on — expect hooks, rendering, and Next.js specifics.

Q How does React's reconciliation / virtual DOM work?

On state change React builds a new virtual tree and diffs it against the previous one, computing the minimal set of real DOM mutations. Diffing is heuristic and O(n): it assumes different element types produce different trees, and uses `key` to match children across renders.

Q Why do lists need a stable `key`, and why is index-as-key a trap?

Keys let React match elements between renders. Using the array index breaks when items are inserted/reordered/removed — React reuses the wrong DOM node and component state, causing subtle bugs (wrong input values, misplaced focus). Use a stable domain id.

Q Explain `useEffect`: dependency array, cleanup, and common mistakes.

Runs after render. The deps array controls when it re-runs: `[]` = once on mount; `[x]` = when `x` changes; omitted = every render. Return a cleanup function for subscriptions/timers — it runs before the next effect and on unmount.

- Missing deps → stale closures reading old state.
- Objects/functions in deps → new identity each render → infinite loops.
- Using effects for derived data that could just be computed in render.

> **Modern angle:** Say you reach for effects only for real external synchronization (fetch, subscriptions, DOM), not for transforming props into state. This is exactly the "You Might Not Need an Effect" guidance.

Q `useMemo` vs `useCallback` — and when NOT to use them.

`useMemo` caches a computed value; `useCallback` caches a function reference (it's `useMemo` for functions). Use them to skip expensive recomputation or to keep stable references for memoized children / effect deps. Don't sprinkle them everywhere — they add overhead and complexity, and premature memoization often costs more than it saves.

Q What problem does `useRef` solve that state doesn't?

A mutable container that persists across renders **without triggering a re-render** when changed. Two uses: referencing DOM nodes, and holding mutable values (timer ids, previous values, instance-like flags) that shouldn't cause updates.

Q Rules of Hooks — what are they and why?

Call hooks only at the top level (never in conditions/loops) and only from React functions. React tracks hook state by **call order**, so conditional calls would misalign state between renders.

Q How do you manage shared/global state without reaching for Redux immediately?

- **Local** — `useState` / `useReducer`.
- **Cross-tree, low-frequency** — Context (theme, auth, locale).
- **Server state** — React Query / SWR (caching, revalidation, dedup) rather than hand-rolled global stores.
- **Complex client state** — Zustand / Redux Toolkit.

Key insight for interviews: distinguish **server state** from **client state** — most "global state" is actually cached server data.

Q Controlled vs uncontrolled components?

Controlled: React state is the source of truth (`value` + `onChange`). Uncontrolled: the DOM holds the value, read via a ref. Controlled is default for validation/dynamic UI; uncontrolled suits simple or performance-sensitive forms.

Q How do you prevent unnecessary re-renders?

- `React.memo` for pure presentational children.
- Stable references via `useCallback` / `useMemo`.
- Lift state down / colocate it so fewer components subscribe.
- Split context or use selectors to avoid broad invalidation.

Q CSR vs SSR vs SSG vs ISR in Next.js — trade-offs?

Mode	Rendered	Best for
CSR	In browser	Private dashboards, highly interactive
SSR	Per request on server	Personalized + SEO-sensitive
SSG	At build time	Mostly static content, fastest
ISR	Static + background revalidate	Large, semi-fresh content

SSR/SSG improve first paint and SEO; CSR shifts work to the client.

Q App Router: Server Components vs Client Components?

Server Components (default) run on the server, ship zero JS, and can fetch data / touch the DB directly. **Client Components** (`"use client"`) run in the browser and are needed for state, effects, and event handlers. The pattern: keep pages as Server Components, push interactivity down into small Client leaves.

> **Wingos-relevant:** This directly serves "intuitive interfaces + performance" — you can articulate shipping less JS to classroom devices by keeping most of the tree server-rendered.

Q How does data fetching / caching work in the App Router?

`fetch` is extended with caching semantics — cached by default, opt into revalidation with `{ next: { revalidate: N } }` or `cache: 'no-store'` for dynamic. Server Components `await` data directly; Suspense boundaries stream UI as data resolves.

3 Node.js & Server-Side

Backend fundamentals — concurrency, async patterns, and running Node in production.

Q Node is single-threaded — how does it handle thousands of concurrent connections?

Non-blocking I/O plus the event loop. Node offloads I/O (network, file, DNS) to the OS / libuv thread pool and registers callbacks; the main thread never blocks waiting. So one thread juggles many in-flight operations. CPU-bound work is the exception — it does block the loop.

Q How do you handle CPU-intensive work without blocking?

- **Worker Threads** for CPU work sharing memory in-process.
- **Child processes / cluster** to use multiple cores.
- Offload to a queue + separate worker service for heavy jobs.

Q Callbacks → Promises → async/await. What did each fix?

Callbacks caused "callback hell" and messy error handling. Promises made async composable with chaining and unified errors via `.catch`. `async/await` gives synchronous-looking control flow with normal `try/catch`. Use `Promise.all` for concurrency, `Promise.allSettled` when partial failure is OK.

Q How is error handling different for sync vs async, and what about unhandled rejections?

Sync errors propagate via `throw` / `try-catch`. In async code, a rejected Promise not awaited/caught becomes an **unhandled rejection**. Wrap awaits in `try/catch`, use centralized error middleware in Express, and treat an uncaught exception as a signal to log and restart the process rather than continue in a bad state.

Q What is middleware in Express, and how does the chain work?

Functions with signature `(req, res, next)` that run in order. Each either responds or calls `next()` to pass control. Used for logging, auth, body parsing, validation. Error middleware has four args `(err, req, res, next)` and is registered last.

Q Streams — what are they and why do they matter?

Interfaces for processing data in chunks instead of loading it all in memory — readable, writable, duplex, transform. Essential for large files/uploads and proxying: you `pipe()` a read stream to a write stream and keep memory flat regardless of size.

Q How do you run Node in production reliably?

- Run multiple instances (cluster / container replicas) behind a load balancer to use all cores.
- Process manager or orchestrator restarts on crash; health checks for liveness/readiness.
- Graceful shutdown: stop accepting connections, drain in-flight, close DB pool on `SIGTERM`.
- Structured logging + centralized config via env vars.

Q How do you keep secrets and config out of code?

Environment variables injected at deploy, a secrets manager for sensitive values, `.env` only for local dev and git-ignored. Never commit keys; validate required env at boot and fail fast if missing.

4 Python & FastAPI

The listing accepts Node and/or Python/FastAPI — know enough to speak to it credibly.

Q Why FastAPI over Flask/Django for a new API?

Native async support, automatic request/response validation via **Pydantic**, and auto-generated OpenAPI/Swagger docs from type hints. It's built on Starlette (ASGI) and Pydantic, so you get high throughput and type safety with little boilerplate.

Q What does Pydantic give you?

Runtime validation and parsing from type-annotated models. Incoming JSON is coerced and validated against the schema; invalid data yields a clear 422 automatically. It's the contract layer between the wire and your Python objects.

```
class TaskIn(BaseModel):
    title: str
    priority: int = 1

@app.post("/tasks")
async def create(task: TaskIn):
    return {"ok": True, "title": task.title}
```

Q Explain `async def` in FastAPI and the GIL.

An `async` endpoint runs on the event loop; `await` yields during I/O so other requests proceed. The **GIL** means only one thread executes Python bytecode at a time, so async helps I/O-bound work, not CPU-bound. Blocking calls inside an async route stall the loop — run them in a threadpool (`run_in_threadpool`) or use async drivers.

Q How does dependency injection work in FastAPI?

The `Depends()` system resolves and injects dependencies (DB sessions, auth, settings) per request, with automatic cleanup for generator dependencies. It keeps endpoints thin and makes shared concerns (a DB session, current user) testable and reusable.

Q Sync vs async DB access in Python — what's the catch?

To stay non-blocking you need async drivers (`asyncpg` , async SQLAlchemy). Mixing a synchronous driver into an async route blocks the loop. If you must use sync libraries, run them in a threadpool so the event loop stays responsive.

Q List vs tuple vs set vs dict — quick contrast.

Type	Ordered	Mutable	Use
list	yes	yes	ordered sequence
tuple	yes	no	fixed record, hashable key
set	no	yes	uniqueness, membership
dict	insertion	yes	key→value, O(1) lookup

Q What are decorators?

A function that wraps another to extend its behaviour without changing its body — the `@decorator` syntax. Used for routing (`@app.get`), auth, caching, timing. Under the hood it's just `fn = decorator(fn)` .

5 Databases — SQL & NoSQL

PostgreSQL/MySQL/MongoDB are all listed — expect modelling, indexing, and transaction questions.

Q SQL vs NoSQL — how do you choose?

Relational (Postgres/MySQL): structured, related data; strong consistency; complex queries and joins; ACID transactions. **Document** (Mongo): flexible/evolving schema, denormalized aggregates, horizontal scale, read-heavy access by a single key. Choice follows the access patterns and consistency needs, not hype.

Q What are ACID properties?

Atomicity (all-or-nothing), **Consistency** (valid state to valid state), **Isolation** (concurrent txns don't interfere), **Durability** (committed data survives crashes). Postgres/MySQL(InnoDB) give these guarantees.

Q How do indexes work, and what's the downside?

Usually a B-tree that turns full scans into fast lookups on indexed columns (and helps sorting/ranges). Downside: extra storage and slower writes, since every insert/update maintains the index. Index the columns you filter/join/sort on — not everything.

Q What is an N+1 query problem and how do you fix it?

Fetching a list (1 query), then a related row per item (N queries) — often hidden behind an ORM/lazy loading. Fix with a join / eager loading, or batch the follow-ups (an `IN` query / DataLoader). Very common source of slow endpoints.

Q How do you find and fix a slow query?

Run `EXPLAIN ANALYZE` to see the plan and where time goes (seq scans, bad joins). Add or fix indexes, reduce selected columns, rewrite the query, and check statistics. Measure before and after rather than guessing.

Q Normalization vs denormalization?

Normalization removes redundancy by splitting into related tables — fewer anomalies, more joins. Denormalization duplicates data to avoid joins for read speed, at the cost of update complexity. OLTP tends normalized; read-heavy/analytics or document stores lean denormalized.

Q How do transactions and isolation levels prevent concurrency bugs?

A transaction groups statements atomically. Isolation levels (Read Committed → Serializable) trade performance for protection against dirty reads, non-repeatable reads, and phantoms. Postgres defaults to Read Committed; bump to Serializable when correctness under concurrency matters.

Q Mongo specifics: embedding vs referencing?

Embed when data is accessed together and bounded (order + line items) — one read, no join. **Reference** when data is shared, unbounded, or updated independently (users ↔ posts). Model around your dominant query, and remember documents have a 16 MB cap.

6 API Design

"Build features across UI, API and database" — API design questions are very likely.

Q What makes a REST API well-designed?

- Resource-based nouns (`/students/42/grades`), not verbs.
- HTTP methods for semantics: GET/POST/PUT/PATCH/DELETE.
- Correct status codes (200/201/204, 400/401/403/404, 409, 422, 500).
- Stateless requests; consistent pagination, filtering, versioning.

Q PUT vs PATCH vs POST, and what does idempotent mean?

POST creates (not idempotent). **PUT** replaces the whole resource (idempotent). **PATCH** partially updates. **Idempotent** = repeating the same request yields the same server state — matters for safe retries. GET/PUT/DELETE are idempotent; POST generally isn't.

Q REST vs GraphQL — when would you pick GraphQL?

GraphQL lets clients request exactly the fields they need in one round trip — good for varied clients and deeply nested data, avoiding over/under-fetching. Costs: caching is harder, and you must guard query depth/complexity. REST is simpler and cache-friendly for straightforward resource access.

Q How do you handle authentication vs authorization in an API?

AuthN = who you are (verify credentials, issue a token/session). **AuthZ** = what you can do (roles/permissions checked per request). Common pattern: short-lived JWT/access token + refresh token, or server sessions; enforce authorization in middleware, never trust the client.

Q How do you version an API and evolve it without breaking clients?

URL versioning (`/v1/…`) or header-based. Add fields rather than removing; make new fields optional; deprecate with a window and communicate. Avoid breaking response shapes silently.

7 Git · Docker · CI/CD

Explicitly required. Expect practical workflow and containerization questions.

Q Explain your Git branching workflow.

Feature branches off `main`, small focused commits, PR with review and green CI before merge. Trunk-based / short-lived branches to reduce merge pain. Be ready to explain merge vs rebase and resolving conflicts.

Q Merge vs rebase — trade-offs?

Merge preserves history and creates a merge commit — safe for shared branches. **Rebase** replays commits onto a new base for linear history — cleaner, but rewrites history, so never rebase branches others have pulled. "Golden rule: don't rebase public/shared branches."

Q How do you undo things safely — `revert` vs `reset`?

`git revert` creates a new commit undoing a change — safe on shared branches. `git reset` moves the branch pointer (`--soft` keeps changes staged, `--hard` discards) — rewrites history, use locally. Mention `git reflog` as the recovery net.

Q What problem does Docker solve, and image vs container?

Consistent environments — "works on my machine" disappears because the app ships with its dependencies. An **image** is the immutable build artifact; a **container** is a running instance of it. Containers share the host kernel, so they're lighter than VMs.

Q How do you write an efficient Dockerfile?

- Order layers by change frequency; copy `package.json` and install **before** copying source so deps cache.
- **Multi-stage builds** — build with full toolchain, ship a slim runtime image.
- Small base images (alpine/slim), `.dockerignore`, non-root user, pinned versions.

> **Strong signal:** Mentioning layer caching + multi-stage builds shows you've actually optimized images, not just written one.

Q What does a CI/CD pipeline do, and what stages do you include?

CI automatically builds and tests every push; CD deploys passing builds. Typical stages: install → lint/typecheck → unit/integration tests → build → build & push image → deploy (staging → prod). Fast feedback and preventing broken code from merging are the point.

Q How do you deploy without downtime?

Rolling or blue-green deploys behind a load balancer, health checks before shifting traffic, and the ability to roll back to the previous image. Run DB migrations backward-compatibly so old and new versions coexist briefly.

8 LLM Integration

Wingos is "AI-powered" and asks to "integrate LLM-based services where they add real value." This section differentiates you.

Q How do you integrate an LLM into a product responsibly?

Treat the LLM as an external, unreliable, non-deterministic service: validate and constrain its output, handle latency/timeouts/rate limits, stream responses for UX, and keep a fallback. Add it where it genuinely helps (drafting, explanation, grading assistance) rather than as a gimmick — mirroring the job's "real value" framing.

Q What is RAG and when do you use it?

Retrieval-Augmented Generation: retrieve relevant documents (usually via vector similarity search over embeddings) and pass them as context so the model answers from your data, not just its training. Use it to ground answers, cite sources, and reduce hallucination — e.g. answering from a school's own curriculum material.

Q What are embeddings and a vector database?

Embeddings map text to high-dimensional vectors where semantic similarity ≈ closeness. A vector DB (pgvector, Pinecone, Qdrant) stores them and does fast nearest-neighbour search. Foundation for semantic search and RAG.

Q How do you get structured, reliable output from an LLM?

- Request JSON / use structured-output or function-calling modes.
- Validate against a schema (Pydantic/Zod) and retry on failure.
- Constrain with clear system prompts and examples; keep temperature low for deterministic tasks.

Q How do you handle cost, latency, and prompt-injection risk?

Cache repeated calls, pick model size per task, cap tokens, and stream to hide latency. For safety: never trust model output as commands, sanitize what you interpolate into prompts, isolate untrusted user content, and keep the model away from privileged actions without validation. Relevant given student-facing use.

Q What causes hallucinations and how do you reduce them?

The model predicts plausible text without a notion of truth. Mitigate with RAG grounding, asking it to cite/quote sources, lower temperature, constrained outputs, and a human-in-the-loop for high-stakes content (e.g. anything shown to students/ teachers).

9 Performance · Reliability · Security

The role explicitly says "own platform performance, reliability and security." Expect all three.

Q How do you improve frontend performance / Core Web Vitals?

- Code-split & lazy-load; ship less JS (server components).
- Optimize images (modern formats, correct sizing, lazy).
- Memoize expensive work; virtualize long lists.
- Cache/CDN static assets; reduce layout shift (CLS) and blocking resources.

Q Backend is slow — how do you diagnose and fix it?

Measure first: APM/tracing to find the slow span. Usual suspects: unindexed/N+1 queries, missing caching, synchronous blocking work, chatty external calls. Fixes: add indexes, cache hot reads, batch, move heavy work to background jobs, add connection pooling.

Q Where and how do you add caching?

Layers: browser/CDN for static, application cache (Redis) for hot data/sessions, DB query cache. The hard part is invalidation — use TTLs, cache keys tied to data, and invalidate on write. Cache reads that are frequent and rarely change.

Q Name the top web security risks and defenses.

- **SQL injection** → parameterized queries / ORM, never string-concatenate SQL.
- **XSS** → escape/encode output, sanitize input, CSP; React escapes by default (beware `dangerouslySetInnerHTML`).
- **CSRF** → tokens / SameSite cookies.
- **Broken auth** → hashed passwords (bcrypt/argon2), secure session/JWT handling.
- Least privilege, HTTPS everywhere, keep dependencies patched.

> **Classroom context:** Handling minors' data — be ready to mention data minimization, access control, and privacy-by-design. It fits SnT's "Security, Reliability and Trust" focus.

Q How do you make a system reliable / observable?

Health checks, structured logging, metrics and tracing (the three pillars), alerting on SLOs, graceful degradation, retries with backoff for transient failures, and timeouts/circuit breakers so one failing dependency doesn't cascade.

Q How do you validate and sanitize user input across the stack?

Validate on the server regardless of client checks — client validation is UX only. Use schema validation (Zod/Pydantic), whitelist allowed values, enforce types/lengths, and encode on output. Never trust anything from the client.

Q How do you store passwords and manage sessions/tokens securely?

Hash with a slow salted algorithm (bcrypt/argon2) — never encrypt or store plain-text. Tokens: short-lived access + refresh, store securely (httpOnly cookies mitigate XSS token theft), rotate and revoke on logout, and always transport over HTTPS.

10 Accessibility (a11y)

The role stresses "intuitive and accessible interfaces for teachers and students" — a likely differentiator most candidates skip.

Q How do you build accessible interfaces?

- Semantic HTML first (`<button>`, `<nav>`, headings) — it gives keyboard and screen-reader behaviour for free.
- Keyboard navigability and visible focus states.
- ARIA only to fill gaps semantics can't; labels for inputs.
- Sufficient colour contrast; don't rely on colour alone.
- Alt text; respect reduced-motion preferences.

Q What is WCAG and what does "accessible" mean in practice?

WCAG is the accessibility standard, organized around **Perceivable, Operable, Understandable, Robust**, with A/AA/AAA levels (AA is the common target). Practically: usable by keyboard, screen readers, and people with low vision or motor limitations. For a school product used by diverse students, this is a genuine product requirement, not a checkbox.

Q How would you test accessibility?

Automated tools (axe, Lighthouse) catch the low-hanging fruit, but manual testing matters: navigate the whole flow with only the keyboard, test with a screen reader, check contrast, and verify focus order. Automated checks catch a minority of real issues.